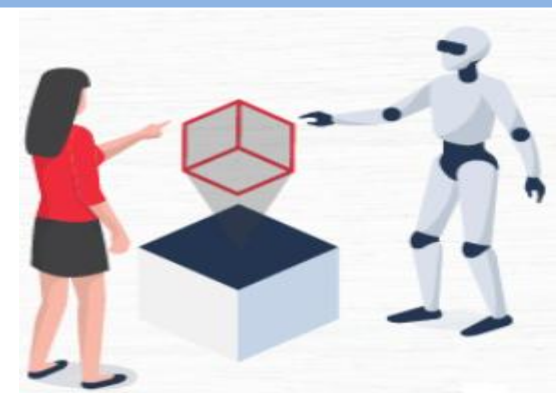
ARTIFICIAL INTELLIGENCE

Unit-III: Knowledge Representation and Reasoning



Unit-III: Knowledge Representation and Reasoning

- Propositional Logic
- First-Order Logic, Forward Chaining and Backward Chaining
- Introduction to Probabilistic Reasoning
- Bayes Theorem
- Knowledge Representation Issues
- Other Knowledge Representation Schemes

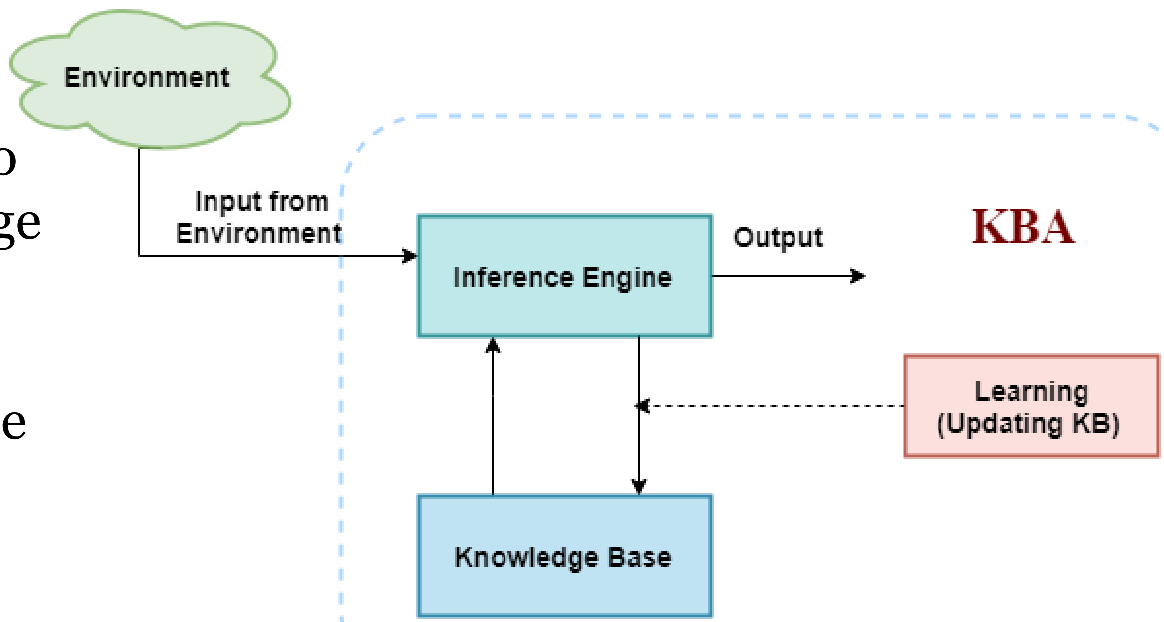
Knowledge-Based Agent in Artificial intelligence

- An intelligent agent needs **knowledge** about the real world for taking decisions and **reasoning** to act efficiently.
- Knowledge-based agents are those agents who have the capability of maintaining an **internal state** of knowledge, **reason** over that knowledge, **update** their knowledge after observations and take actions.
- These agents can **represent** the world with some formal representation and act intelligently.

Knowledge-Based Agent in Artificial Intelligence

- Knowledge-based agents are composed of two main parts:
  - **Knowledge-base and**
  - **Inference system**
- knowledge-based agent (KBA) take input from the environment by perceiving the environment.

- The input is taken by the **inference** engine of the agent and which also **communicate** with KB to decide as per the knowledge store in KB.
- The learning element of KBA regularly **update**s the KB by learning new knowledge

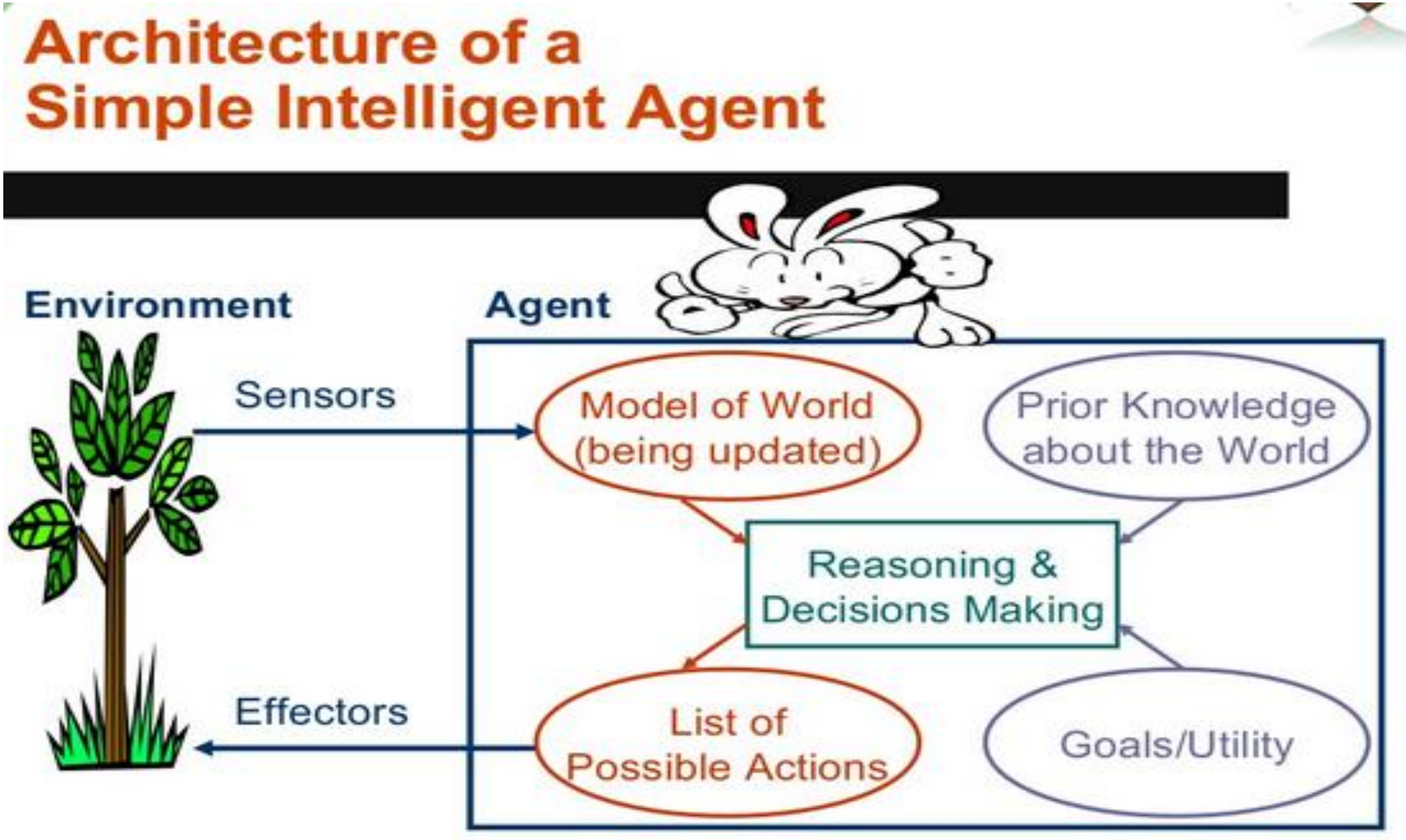


Knowledge Representation

- Knowledge Representation is a field of artificial intelligence that is concerned with presenting **real-world information** in a form that the computer can 'understand' and use to 'solve' **real-life problems** or 'handle' **real-life tasks**.

- The ability of machines to think and act like humans such as understanding, interpreting and reasoning constitute knowledge representation. It is related to designing agents that can think and ensure that such thinking can constructively contribute to the agent's behavior.

Knowledge Representation



Knowledge Representation

- In simple words, knowledge representation allows machines to behave like humans by empowering an AI machine to learn from available information, experience or experts.
- However, it is important to choose the right type of knowledge representation if you want to ensure business success with AI.

Propositional Logic

- Propositional logic (PL) is the simplest form of logic where all the statements are made by **propositions**.
- A proposition is a declarative statement which is either **true** or **false**.
- It is a technique of knowledge representation in **logical** and **mathematical** form.

Example:

- a) It is Sunday
- b) The Sun rises from West (False proposition)
- c) 3+3= 7 (False proposition)
- d) 5 is a prime number.

Propositional Logic (PL)

- Propositional logic is also called **Boolean** logic as it works on 0 and 1.
- In PL, we use symbolic variables to represent the logic, and we can use any symbol for a representing a proposition, such A, B, C, P, Q, R, etc.
- Propositions can be either **true** or **false**, but it cannot be both.
- Propositional logic consists of an **object**, **relations** or function, and **logical connectives**.
- These connectives are also called logical **operators**
- The propositions and connectives are the basic elements of the propositional logic.

Propositional Logic

- Connectives can be said as a logical operator which connects two sentences.
- A proposition formula which is always true is called **tautology**, and it is also called a valid sentence.
- A proposition formula which is always false is called **Contradiction**.
- A proposition formula which has both true and false values is called
- Statements which are questions, commands, or opinions are not propositions such as "**Where is Rohini**", "**How are you**", "**What is your name**", are not propositions.

Propositional Logic Formulae

Formulae in propositional logic are defined as follows:

Any propositional variable is a formula.

- **Conjunction**: If and are formulas, then (read as AND) is a formula.
- **Disjunction**: If and are formulas, then (read as OR) is a formula.
- **Negation**: If is a formula then (read as NOT) is a formula.

Caution on "OR" operator

- When we say "Either OR is true", we mean that **at least** one of the two propositions , is true. It could be that both of and are true. (See the definition in the textbook.)
- This is different from the **exclusive OR** (written as **XOR** or). For any propositions and , means "either or , but not both".

Propositional Logic

Example

- Consider the following two propositions:
  - It is rainy today.
  - It is sunny today.

Then:

$p \wedge q$	**It is rainy and sunny today.**
$p \vee q$	It is rainy or sunny today.
$p \wedge (\neg q)$	It is rainy and not sunny today.

Truth table - Propositional Logic

- We evaluate propositional formulae using *truth tables*.
- For any given proposition formula depending on **several** propositional variables, we can draw a truth table considering **all** possible **combinations** of **boolean** values that the variables can take, and in the table we evaluate the resulting **boolean** value of the proposition formula for each combination of boolean values.

Truth Table for AND

Truth Table for AND

p	q	$p \wedge q$
T	T	T
T	F	F
F	T	F
F	F	F

- Each row represents some kind of a *situation*. For example, the top most row represents the situation when propositions p and q and are both "true".
- Then we conclude that the formula $p \wedge q$ in this situation is also "true".

Logicians call these situations *models*. In this case, a Logician would say that the truth assignment $p \rightarrow$ true, $q \rightarrow$ true is a model of the formula $p \wedge q$

Truth Table for AND

- In other words, if you imagine a situation wherein the proposition p is true and the proposition q is true then in that situation formula is $p \wedge q$ true.
- To avoid confusion let us use the term "situations" and "models".
- We will formalize models later for first-order (predicate) logic.

Truth Table for OR

Truth Table for NOT

Truth Table for OR

p	q	$p \vee q$
T	T	T
T	F	T
F	T	T
F	F	F

Truth Table for NOT

p	$\neg p$
T	F
F	T

Truth Table for Compound Formulae

XOR is an important derived connective that is defined in terms of $\wedge, \vee, \neg$.
It has the interpretation of "either-or": i.e, either p or q, but not both.
p XOR q is the formula $(p \wedge \neg q) \vee (\neg p \wedge q)$.
Its truth table can be written as below:

p	q	$p \wedge \neg q$	$\neg p \wedge q$	p **XOR** q
T	F	T	F	T
T	T	F	F	F
F	T	F	T	T
F	F	F	F	F

We will go through few more examples of truth tables in the book.

Other examples of derived connectives are:

- **Implication** $p \Rightarrow q$ is defined as $\neg p \vee q$ (we will study this in detail next week).
- **Equivalence** $p \Leftrightarrow q$ is defined as $(p \wedge q) \vee (\neg p \wedge \neg q)$.
- **Nand** (a.k.a not of and) $p \text{ NAND } q$ is defined as $\neg(p \wedge q)$.
- **NOR** (a.k.a not of or) $p \text{ NOR } q$ is defined as $\neg(p \vee q)$
- **XOR** (exclusive or) $p \text{ XOR} q$ is defined as $(p \vee q) \wedge (\neg(p \wedge q))$.

First-Order Logic

- In Propositional logic, we have seen that how to represent statements using propositional logic.
- But unfortunately, in propositional logic, we can only represent the facts, which are either true or false.
- PL is not sufficient to represent the complex sentences or natural language statements. The propositional logic has very limited expressive power. Consider the following sentence, which we cannot represent using PL logic.
- **"Some humans are intelligent", or**
- **"Sachin likes cricket."**

First-Order Logic

- To represent the above statements, PL logic is not sufficient, so we required some more powerful logic, such as first-order logic.

First-Order logic:

- First-order logic is another way of knowledge representation in artificial intelligence.
- It is an extension to propositional logic.
- FOL is sufficiently expressive to represent the **natural language** statements in a concise way.

First-Order Logic

- First-order logic is also known as **Predicate logic or First-order predicate logic**.
- First-order logic is a powerful language that develops information about the objects in a more **easy way** and can also express the **relationship** between those objects.

- First-order logic (like natural language) does **not** only assume that the world contains **facts** like propositional logic but also assumes the following things in the world:

First-Order Logic

  - **Objects:** A, B, people, numbers, colors, wars, theories, squares, pits, wumpus, ……
  - **Relations: It can be unary relation such as:** red, round, is adjacent, **or n-any relation such as:** the sister of, brother of, has color, comes between
  - **Function:** Father of, best friend, third inning of, end of, ……
- As a natural language, first-order logic also has two main parts:
  - **Syntax**
  - **Semantics**

First-Order Logic

- **Syntax of First-Order logic:**
- The syntax of FOL determines which collection of symbols is a logical expression in first-order logic.
- The basic syntactic elements of first-order logic are symbols. We write statements in short-hand notation in FOL.

Basic Elements of First-order logic:

Following are the basic elements of FOL syntax:

Constant	1, 2, A, John, Mumbai, cat,....
Variables	x, y, z, a, b,....
Predicates	Brother, Father, >,....
Function	sqrt, LeftLegOf, ...
Connectives	∧, ∨, ¬, ⇒, ⇔
Equality	==
Quantifier	∀, ∃

First-Order Logic

- **Atomic sentences:**
- Atomic sentences are the most basic sentences of first-order logic. These sentences are formed from a predicate symbol followed by a parenthesis with a sequence of terms.
- We can represent atomic sentences as **Predicate** (term1, term2, ..., term n).

- **Example:** Ravi and Ajay are brothers: => Brothers(Ravi, Ajay).

  Chinky is a cat: => cat (Chinky).

First-Order Logic

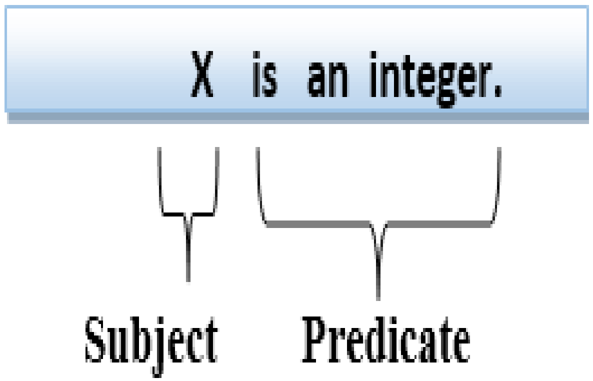
- **Complex Sentences:**
- Complex sentences are made by combining atomic sentences using connectives.

First-order logic statements can be divided into two parts:

- **Subject:** Subject is the main part of the statement.
- **Predicate:** A predicate can be defined as a relation, which binds two atoms together in a statement.

First-Order Logic

- **Consider the statement: "x is an integer."**, it consists of two parts, the first part x is the subject of the statement and second part "is an integer," is known as a predicate.



Forward Chaining and Backward Chaining

- In artificial intelligence, forward and backward chaining is one of the important topics, but before understanding forward and backward chaining lets first understand that from where these two terms came.

Inference engine:

- The inference engine is the component of the intelligent system in artificial intelligence, which applies **logical rules** to the **knowledge base** to infer new information from known facts.
- The first inference engine was part of the expert system. Inference engine commonly proceeds in two modes, which are:
- **Forward chaining**
- **Backward chaining**

Horn Clause and Definite Clause

- Horn clause and definite clause are the forms of sentences, which enables knowledge base to use a more restricted and efficient inference algorithm. Logical inference algorithms use forward and backward chaining approaches, which require KB in the form of the first-order definite clause.
- **Definite clause:** A clause which is a disjunction of literals with **exactly one positive literal** is known as a definite clause or strict horn clause.
- **Horn clause:** A clause which is a disjunction of literals with **at most one positive literal** is known as horn clause. Hence all the definite clauses are horn clauses.
- **Example: ($\neg$ p V $\neg$ q V k)**. It has only one positive literal k.
- It is equivalent to $p \wedge q \rightarrow k$.

Forward Chaining

- Forward chaining is also known as a forward deduction or forward reasoning method when using an inference engine.
- Forward chaining is a form of reasoning which start with atomic sentences in the knowledge base and applies inference rules (Modus Ponens) in the **forward direction** to extract more data until a goal is **reached**.
- The Forward-chaining algorithm starts from **known facts**, triggers all rules whose premises are satisfied, and add their conclusion to the known facts. This process repeats until the problem is solved.

Forward Chaining

Properties of Forward-Chaining:

- It is a down-up approach, as it moves from bottom to top.
- It is a process of making a conclusion based on known facts or data, by starting from the initial state and reaches the goal state.
- Forward-chaining approach is also called as data-driven as we reach to the goal using available data.
- Forward -chaining approach is commonly used in the expert system, such as CLIPS, business, and production rule systems.

Forward Chaining

Consider the following famous example which we will use in both approaches:

Example:

- "As per the law, it is a crime for an American to sell weapons to hostile nations. Country A, an enemy of America, has some missiles, and all the missiles were sold to it by Robert, who is an American citizen."
- Prove that **"Robert is criminal."**
- To solve the above problem, first, we will convert all the above facts into first-order definite clauses, and then we will use a forward-chaining algorithm to reach the goal.

Facts Conversion into FOL

- It is a crime for an American to sell weapons to hostile nations. (Let's say p, q, and r are variables)

  American (p) ∧ weapon(q) ∧ sells (p, q, r) ∧ hostile(r) → Criminal(p) ...(1)

- Country A has some missiles. **?p Owns(A, p) ∧ Missile(p)**. It can be written in two definite clauses by using Existential Instantiation, introducing new Constant T1.

  Owns(A, T1) ......(2)

  Missile(T1) ....(3)

- All of the missiles were sold to country A by Robert.

  ?p Missiles(p) ∧ Owns (A, p) → Sells (Robert, p, A) ......(4)

- Missiles are weapons.

  Missile(p) → Weapons (p) ....(5)

- Enemy of America is known as hostile.

  Enemy(p, America) →Hostile(p) ....(6)

- Country A is an enemy of America.

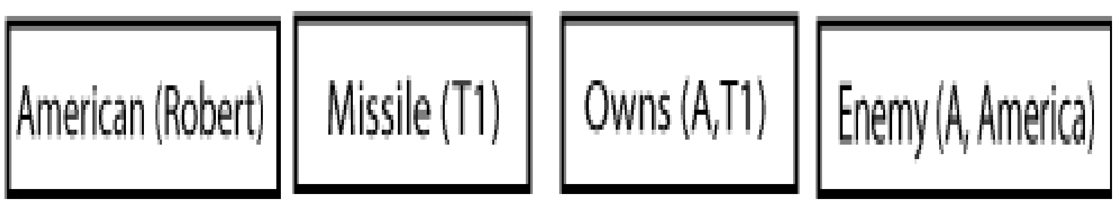
  Enemy (A, America) ....(7)

- Robert is American

  American(Robert). ....(8)

Forward Chaining Proof

- **Step-1:**
- In the first step we will start with the known facts and will choose the sentences which do not have implications, such as: **American(Robert), Enemy(A, America), Owns(A, T1), and Missile(T1)**. All these facts will be represented as below.



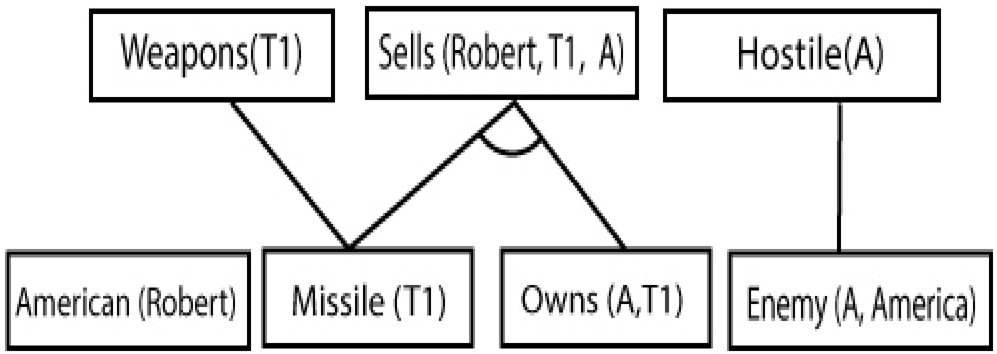
Forward Chaining Proof

Step-2:

- At the second step, we will see those facts which infer from available facts and with satisfied premises.
- Rule-(1) does not satisfy premises, so it will not be added in the first iteration.
- Rule-(2) and (3) are already added.
- Rule-(4) satisfy with the substitution {p/T1}, **so Sells (Robert, T1, A)** is added, which infers from the conjunction of Rule (2) and (3).
- Rule-(6) is satisfied with the substitution(p/A), so Hostile(A) is added and which infers from Rule-(7).

Forward Chaining Proof

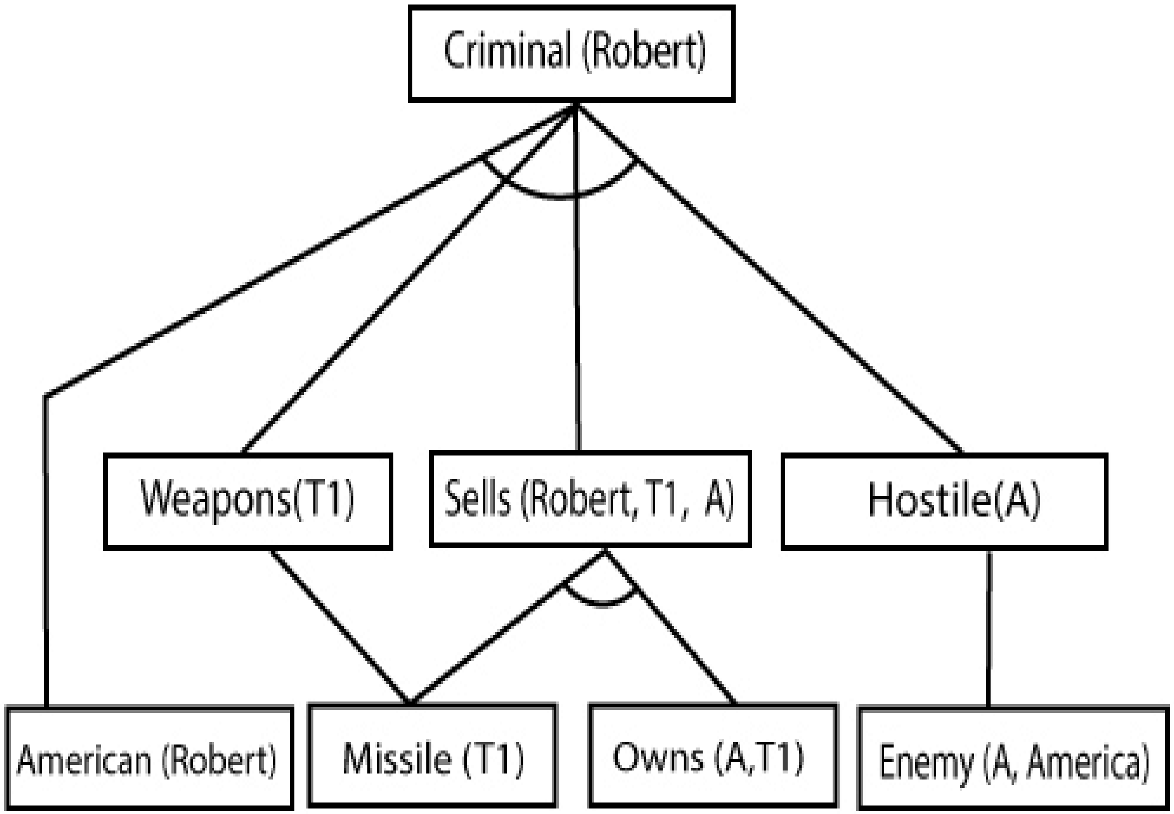
- **Step-2:**



- **Step-3:**
- At step-3, as we can check Rule-(1) is satisfied with the substitution **{p/Robert, q/T1, r/A}, so we can add Criminal(Robert)** which infers all the available facts. And hence we reached our goal statement.

Forward Chaining Proof

- **Step-3:**



- Hence it is proved that Robert is Criminal using forward chaining approach.

Backward Chaining

- Backward-chaining is also known as a backward deduction or backward reasoning method when using an inference engine. A backward chaining algorithm is a form of reasoning, which starts with the goal and works backward, chaining through rules to find known facts that support the goal.
- **Properties of backward chaining:**
- It is known as a top-down approach.
- Backward-chaining is based on modus ponens inference rule.
- In backward chaining, the goal is broken into sub-goal or sub-goals to prove the facts true.

Backward Chaining

- It is called a goal-driven approach, as a list of goals decides which rules are selected and used.
- Backward -chaining algorithm is used in game theory, automated theorem proving tools, inference engines, proof assistants, and various AI applications.
- The backward-chaining method mostly used a **depth-first search** strategy for proof.

Backward Chaining

Example:

- In backward-chaining, we will use the same above example, and will rewrite all the rules.
  - American (p) ∧ weapon(q) ∧ sells (p, q, r) ∧ hostile(r) → Criminal(p) ...(1)
    Owns(A, T1) ....(2)
  - Missile(T1)
  - ?p Missiles(p) ∧ Owns (A, p) → Sells (Robert, p, A) ......(4)
  - Missile(p) → Weapons (p) ....(5)
  - Enemy(p, America) →Hostile(p) ....(6)
  - Enemy (A, America) ....(7)
  - American(Robert). ....(8)

Backward Chaining Proof

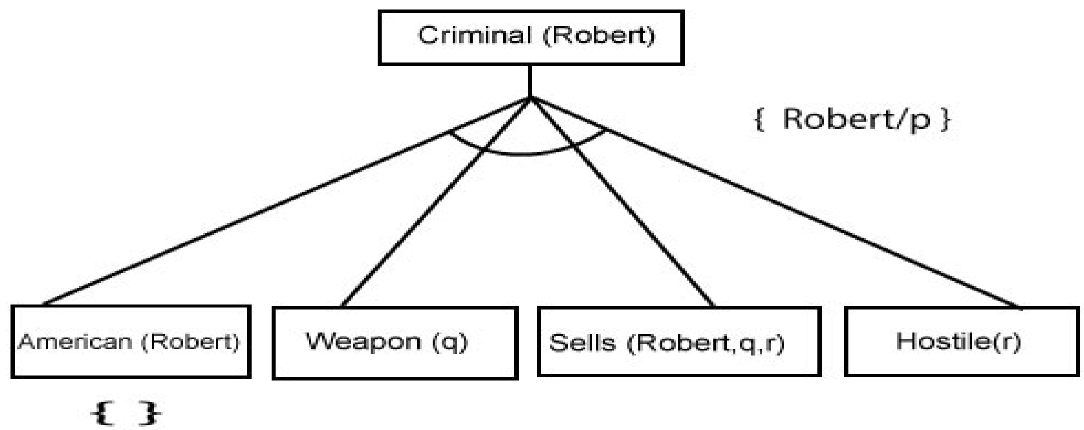
- In Backward chaining, we will start with our goal predicate, which is **Criminal(Robert)**, and then infer further rules.

- **Step-1:**
- At the first step, we will take the goal fact. And from the goal fact, we will infer other facts, and at last, we will prove those facts true. So our goal fact is "Robert is Criminal," so following is the predicate of it.

Criminal (Robert)

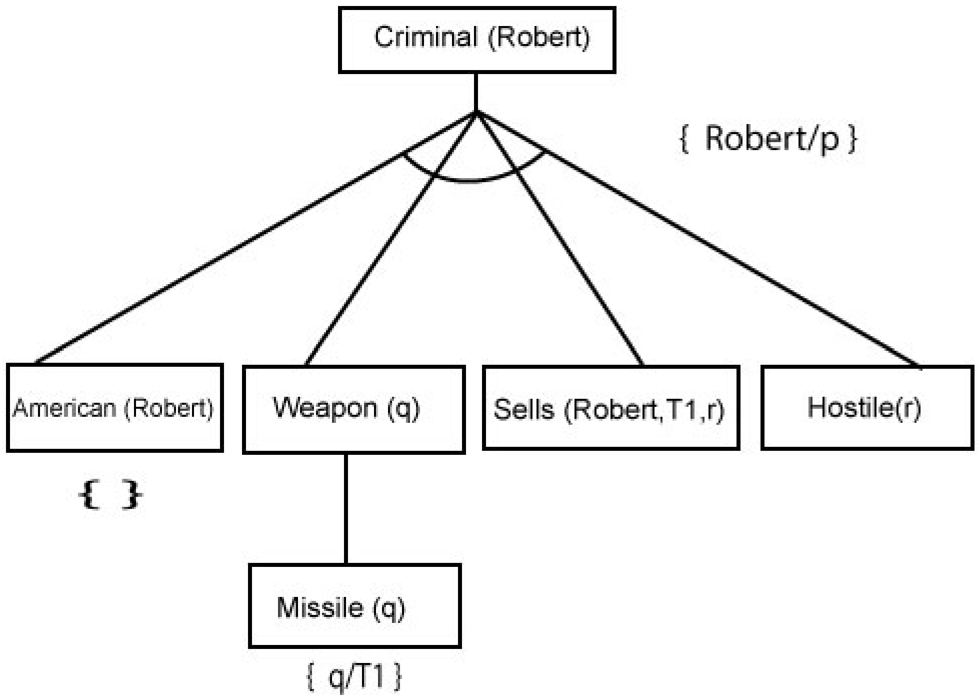
Backward Chaining Proof

- **Step-2:**
- At the second step, we will infer other facts form goal fact which satisfies the rules. So as we can see in Rule-1, the goal predicate Criminal (Robert) is present with substitution {Robert/P}. So we will add all the conjunctive facts below the first level and will replace p with Robert.
- Here we can see American (Robert) is a fact, so it is proved here.



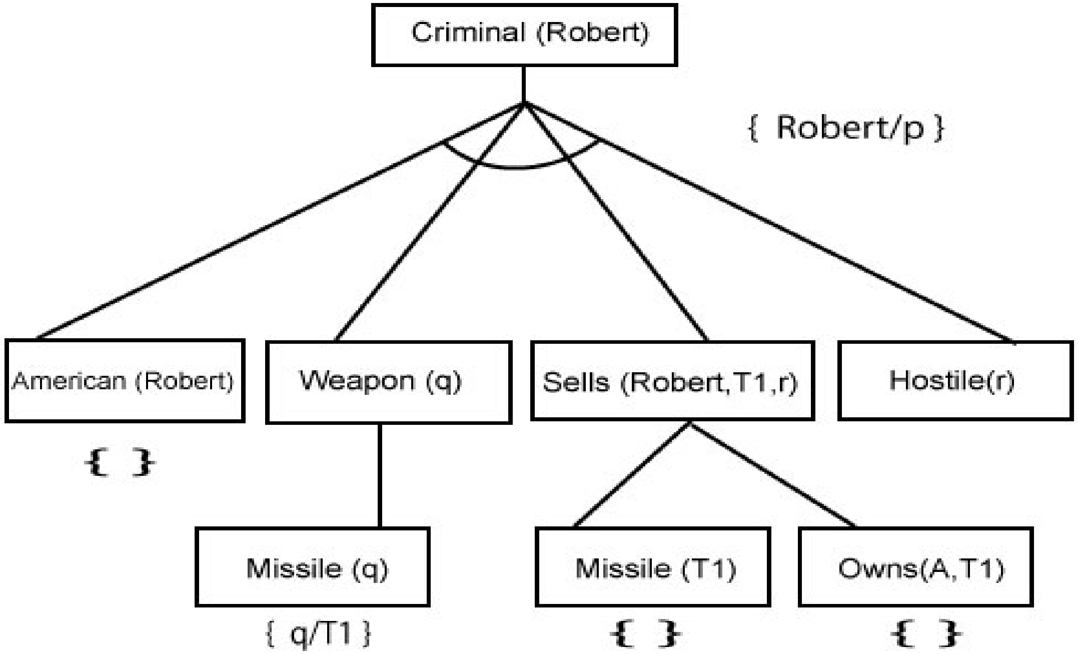
Backward Chaining Proof

- **Step-3:**t At step-3, we will extract further fact Missile(q) which infer from Weapon(q), as it satisfies Rule-(5). Weapon (q) is also true with the substitution of a constant T1 at q.



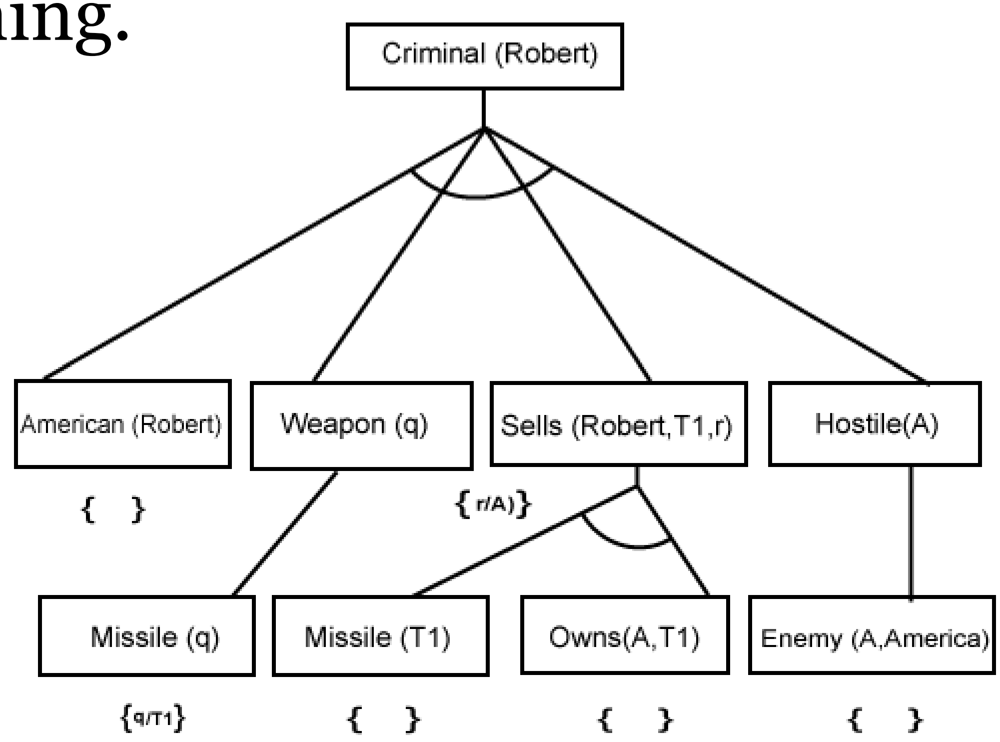
Backward Chaining Proof

- **Step-4:**
- At step-4, we can infer facts Missile(T1) and Owns(A, T1) form Sells(Robert, T1, r) which satisfies the **Rule- 4**, with the substitution of A in place of r. So these two statements are proved here.



Backward Chaining Proof

- **Step-5:**
- At step-5, we can infer the fact **Enemy(A, America)** from **Hostile(A)** which satisfies Rule-6. And hence all the statements are proved true using backward chaining.



Difference between backward chaining and forward chaining

- Forward chaining as the name suggests, start from the known facts and move forward by applying inference rules to extract more data, and it continues until it reaches to the goal, whereas backward chaining starts from the goal, move backward by using inference rules to determine the facts that satisfy the goal.
- Forward chaining is called a **data-driven** inference technique, whereas backward chaining is called a **goal-driven** inference technique.
- Forward chaining is known as the **down-up** approach, whereas backward chaining is known as a **top-down** approach.

Difference between backward chaining and forward chaining

- Forward chaining uses **breadth-first search** strategy, whereas backward chaining uses **depth-first search** strategy.
- Forward and backward chaining both applies **Modus ponens** inference rule.
- Forward chaining can be used for tasks such as **planning, design process monitoring, diagnosis, and classification**, whereas backward chaining can be used for **classification and diagnosis tasks**.

Difference between backward chaining and forward chaining

- Forward chaining can be like an exhaustive search, whereas backward chaining tries to avoid the unnecessary path of reasoning.
- In forward-chaining there can be various ASK questions from the knowledge base, whereas in backward chaining there can be fewer ASK questions.
- Forward chaining is slow as it checks for all the rules, whereas backward chaining is fast as it checks few required rules only.

Forward Chaining Vs. Backward Chaining

S. No.	Forward Chaining	Backward Chaining
1.	Forward chaining starts from known facts and applies inference rule to extract more data unit it reaches to the goal.	Backward chaining starts from the goal and works backward through inference rules to find the required facts that support the goal.
2.	It is a bottom-up approach	It is a top-down approach
3.	Forward chaining is known as data-driven inference technique as we reach to the goal using the available data.	Backward chaining is known as goal-driven technique as we start from the goal and divide into sub-goal to extract the facts.
4.	Forward chaining reasoning applies a breadth-first search strategy.	Backward chaining reasoning applies a depth-first search strategy.
5.	Forward chaining tests for all the available rules	Backward chaining only tests for few required rules.

Forward Chaining Vs. Backward Chaining

S. No.	Forward Chaining	Backward Chaining
6.	Forward chaining is suitable for the planning, monitoring, control, and interpretation application.	Backward chaining is suitable for diagnostic, prescription, and debugging application.
7.	Forward chaining can generate an infinite number of possible conclusions.	Backward chaining generates a finite number of possible conclusions.
8.	It operates in the forward direction.	It operates in the backward direction.
9.	Forward chaining is aimed for any conclusion.	Backward chaining is only aimed for the required data.

Probabilistic Reasoning : Uncertainty

- Till now, we have learned knowledge representation using first-order logic and **propositional logic** with certainty, which means we were sure about the predicates.
- With this knowledge representation, we might write $A \rightarrow B$, which means if A is true then B is true, but consider a situation where we are **not sure** about whether A is **true** or **not** then we cannot express this statement, this situation is called **uncertainty**.
- So to represent **uncertain** knowledge, where we are **not sure** about the **predicates**, we need uncertain reasoning or **probabilistic reasoning**

Uncertainty: Probabilistic Reasoning

Causes of uncertainty:

The following are some leading causes of uncertainty to occur in the real world.

- Information occurred from **unreliable** sources
- Experimental **Errors**
- Equipment **fault**
- Temperature **variation**
- **Climate** change.

Probabilistic Reasoning

- Probabilistic reasoning is a way of knowledge representation where we apply the concept of **probability** to indicate the **uncertainty** in knowledge.
- In probabilistic reasoning, we **combine** **probability theory** with **logic** to handle the uncertainty.
- We use probability in probabilistic reasoning because it provides a way to **handle** the uncertainty that is the result of someone's laziness and ignorance.

Probabilistic Reasoning

- In the real world, there are lots of scenarios, where the certainty of something is not confirmed, such as "**It will rain today**," "behavior of someone for some situations," "A match between two teams or two players."
- These are probable sentences for which we can assume that it will **happen** but **not** sure about it, so here we use probabilistic reasoning.

Probabilistic Reasoning

Need of probabilistic reasoning in AI:

- When there are **unpredictable** outcomes.
- When specifications or **possibilities** of predicates becomes too **large** to handle.
- When an unknown **error** occurs during an experiment.

- In probabilistic reasoning, there are two ways to solve problems with uncertain knowledge:
    - **Bayes' rule**
    - **Bayesian Statistics**

Probabilistic Reasoning

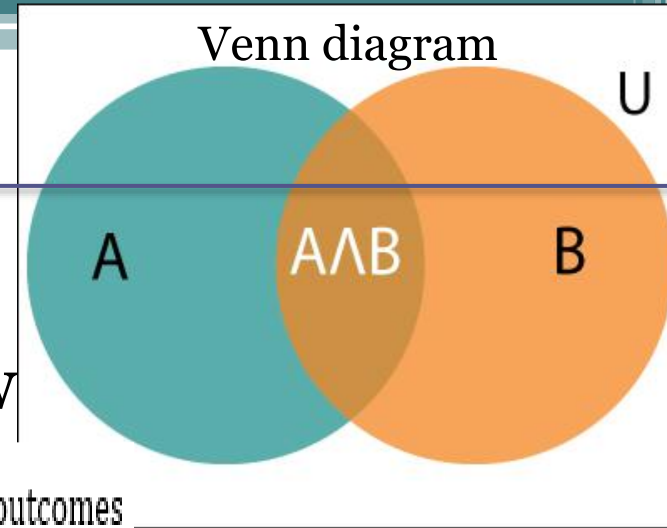
Some common terms:

- **Probability:** Probability can be defined as a chance that an uncertain event will occur. It is the numerical measure of the likelihood that an event will occur. The value of probability always remains between 0 and 1 that represent ideal uncertainties.
  - $0 \leq P(A) \leq 1$

    where P(A) is the probability of an event A.
  - $P(A) = 0$

    indicates total uncertainty in an event A.
  - $P(A) = 1$, *indicates total certainty in an event A.*

Probabilistic Reasoning



We can find the probability of an uncertain event by using the below formula.

$$\text{Probability of occurrence} = \frac{\text{Number of desired outcomes}}{\text{Total number of outcomes}}$$

- Let's suppose, we want to calculate the event A when event B has already occurred, "the probability of A under the conditions of B", it can be written as:

$$P(A|B) = \frac{P(A \wedge B)}{P(B)}$$

Where P(*A*∧*B*)= Joint probability of a and B

P(B)= Marginal probability of B.

Probabilistic Reasoning : Example

- In a class, there are 70% of the students who like English and 40% of the students who likes English and mathematics, and then what is the percent of students those who like English also like mathematics?
- **Solution:**
- Let, A is an event that a student likes Mathematics
- B is an event that a student likes English.

$$P(A|B) = \frac{P(A \wedge B)}{P(B)} = \frac{0.4}{0.7} = 57\%$$

- Hence, 57% are the students who like English also like Mathematics.

Bayes' Theorem

- Bayes' theorem is also known as **Bayes' rule, Bayes' law**, or **Bayesian reasoning**, which determines the **probability** of an **event** with **uncertain** knowledge.
- In probability theory, it relates the **conditional** probability and **marginal** probabilities of two random events.
- Bayes' theorem was named after the British mathematician **Thomas Bayes**.
- The **Bayesian inference** is an application of Bayes' theorem, which is fundamental to Bayesian statistics.

Bayes' Theorem

- It is a way to calculate the value of P(B|A) with the knowledge of P(A|B). *probability of B given A → A|B*
- Bayes' theorem allows updating the probability **prediction** of an event by observing new information of the real world.

Example:

- If cancer corresponds to one's age then by using Bayes' theorem, we can determine the probability of cancer more accurately with the help of age.
- Bayes' theorem can be derived using product rule and conditional probability of event A with known event B:

Bayes' Theorem

As from product rule we can write:

- $P(A \wedge B) = P(A|B)\,P(B)$ or

Similarly, the probability of event B with known event A:

- $P(A \wedge B) = P(B|A)\,P(A)$

Equating right hand side of both the equations, we will get:

$$P(A|B) = \frac{P(B|A)\,P(A)}{P(B)} \quad \text{....(a)}$$

The above equation (a) is called as **Bayes' rule** or **Bayes' theorem**. This equation is basic of most modern AI systems for **probabilistic inference**.

Bayes' Theorem

- It shows the simple relationship between joint and conditional probabilities. Here,
- P(A|B) is known as **posterior**, which we need to calculate, and it will be read as Probability of hypothesis A when we have occurred an evidence B.
- P(B|A) is called the likelihood, in which we consider that hypothesis is true, then we calculate the probability of evidence. P(A) is called the **prior probability**, probability of hypothesis before considering the evidence

Bayes' Theorem

- P(B) is called **marginal probability**, pure probability of an evidence.
- In the equation (a), in general, we can write P (B) = P(A)*P(B|Ai), hence the Bayes' rule can be written as:

$$P(A_i|B) = \frac{P(A_i)*P(B|A_i)}{\sum_{i=1}^{k} P(A_i)*P(B|A_i)}$$

- Where $A_1, A_2, A_3, \ldots\ldots, A_n$ is a set of mutually exclusive and exhaustive events.

Applying Bayes' rule

- Bayes' rule allows us to compute the single term P(B|A) in terms of P(A|B), P(*B*), and P(A). This is very useful in cases where we have a good probability of these three terms and want to determine the fourth one.
- Suppose we want to perceive the effect of some unknown cause, and want to compute that cause, then the Bayes' rule becomes:

$$P(\text{cause}|\text{effect}) = \frac{P(\text{effect}|\text{cause})\, P(\text{cause})}{P(\text{effect})}$$

Applying Bayes' rule

Example-1:

- Question: what is the probability that a patient has diseases meningitis with a stiff neck?
- **Given Data:**
- A doctor is aware that disease meningitis causes a patient to have a stiff neck, and it occurs 80% of the time. He is also aware of some more facts, which are given as follows:
- The Known probability that a patient has meningitis disease is 1/30,000.
- The Known probability that a patient has a stiff neck is 2%.

Applying Bayes' rule

- Let a be the proposition that patient has stiff neck and b be the proposition that patient has meningitis. , so we can calculate the following as:
- P(a|b) = 0.8
- P(b) = 1/30000
- P(a)= .02

$$P(b|a) = \frac{P(a|b)P(b)}{P(a)} = \frac{0.8*(\frac{1}{30000})}{0.02} = 0.001333333.$$

- Hence, we can assume that 1 patient out of 750 patients has meningitis disease with a stiff neck.

Applying Bayes' rule

Example-2:

- *Question: From a standard deck of playing cards, a single card is drawn. The probability that the card is king is 4/52, then calculate posterior probability P(King|Face), which means the drawn face card is a king card.*

$$P(king|face) = \frac{P(Face|king) * P(King)}{P(Face)} \quad ....(i)$$

- P(king): probability that the card is King= 4/52= 1/13
- P(face): probability that a card is a face card= 3/13
- P(Face|King): probability of face card when we assume it is a king = 1
- Putting all values in equation (i) we will get:

$$P(king|face) = \frac{1 * (\frac{1}{13})}{(\frac{3}{13})} = 1/3, \text{ it is a probability that a face card is a king card.}$$

Application of Bayes' theorem in Artificial Intelligence

Following are some applications of Bayes' theorem:

- It is used to calculate the next step of the robot when the already executed step is given.
- Bayes' theorem is helpful in weather forecasting.
- It can solve the Monty Hall problem.

The Monty Hall problem is a counter-intuitive statistics puzzle:
There are 3 doors, behind which are two goats and a car.
You pick a door (call it door A). You're hoping for the car of course.
Monty Hall, the game show host, examines the other doors (B & C) and opens one with a goat. (If both doors have goats, he picks randomly.)

Knowledge Representation Issues

The main objective of knowledge representation is to draw the conclusions from the knowledge, but there are many issues associated with the use of knowledge representation techniques.

Issues that arises while KR techniques:

1. Important Attributes
2. Relationships among Attributes
3. Choosing the granularity of Representation
4. Representing Sets of Objects
5. Finding the Right structures as Needed

Knowledge Representation Issues

- **Important attributes**
  There are two attributes **instance** and **isa** that are of general importance**.** Since these attributes support property of **inheritance**.

2. Relationships among attributes

- The attributes to describe objects are themselves entities they represent.
- The relationship between the attributes of an object, independent of specific knowledge they encode, may hold properties like:
- *Inverses, existence in an isa hierachy, techniques for reasoning about values and single valued attributes.*

Knowledge Representation Issues

3. Choosing the granularity of representation

While deciding the granularity of representation, it is necessary to know the following:

i. What are the primitives and at what level should the knowledge be represented?

ii. What should be the number (small or large) of low-level primitives or high-level facts?

High-level facts may be insufficient to draw the conclusion while Low-level primitives may require a lot of storage.

Knowledge Representation Issues

4. Representing sets of objects.

There are some properties of objects which satisfy the condition of a set together but not as individual;

Example: Consider the assertion made in the sentences:

"There are more sheep than people in Australia", and "English speakers can be found all over the world."

These facts can be described by including an assertion to the sets representing people, sheep, and English.

5. Finding the right structure as needed

To describe a particular situation, it is always important to find the access of right structure. This can be done by selecting an initial structure and then revising the choice.

Knowledge Representation Schemes/**Approaches**

There are four types of Knowledge representation schemes / **Approaches**:

- *Relational* Knowledge
- *Inheritable* Knowledge
- *Inferential* Knowledge *and*
- *Procedural* Knowledge

***Relational* Knowledge**

- provides a framework to compare two objects based on equivalent attributes.
- any instance in which two different objects are compared is a relational type of knowledge.

Other Knowledge Representation Schemes

***Inheritable* Knowledge**

- is obtained from associated objects.
- it prescribes a structure in which new objects are created which may inherit all or a subset of attributes from existing objects.

***Inferential* Knowledge**

- is inferred from objects through relations among objects.
- e.g., a word alone is a simple syntax, but with the help of other words in phrase the reader may infer more from a word; this inference within linguistic is called semantics.

Other Knowledge Representation Schemes

Procedural Knowledge

- A representation in which the control information, to use the knowledge, is embedded in the knowledge itself. For example, computer programs, directions, and recipes; these indicate specific use or implementation;

- Knowledge is encoded in some procedures, small programs that know how to do specific things, how to proceed.

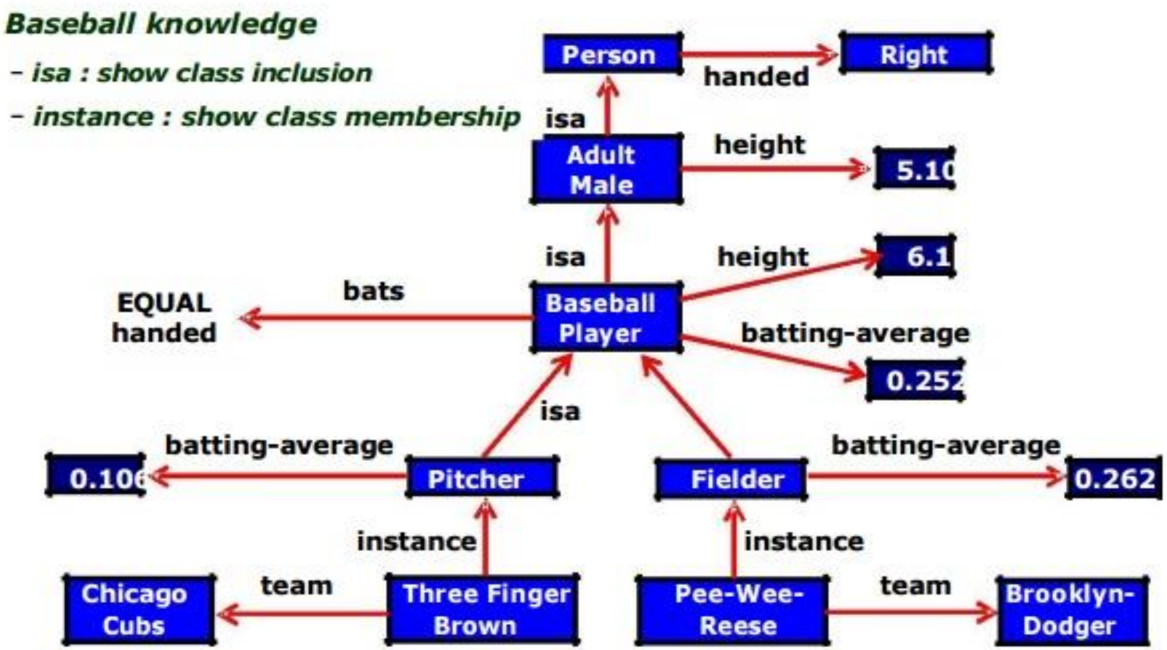


Fig. Inheritable knowledge representation (KR)